

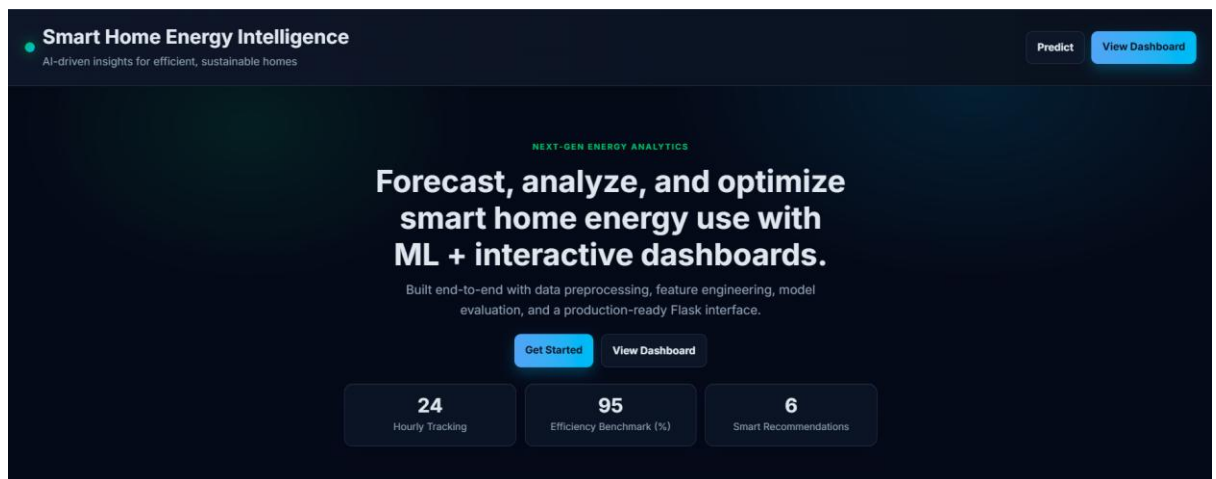
Week 7-8 Documentation

Project: Smart Home Energy Consumption Forecasting

Student: Swapnil Pal

Duration: Week 7-8

1) Objective



- Build a complete, user-facing web application for energy prediction.
- Integrate trained ML model (best_energy_model.pkl) with Flask backend.
- Improve UI/UX to make dashboard more premium, readable, and interactive.
- Validate prediction flow and data visualization APIs end-to-end.

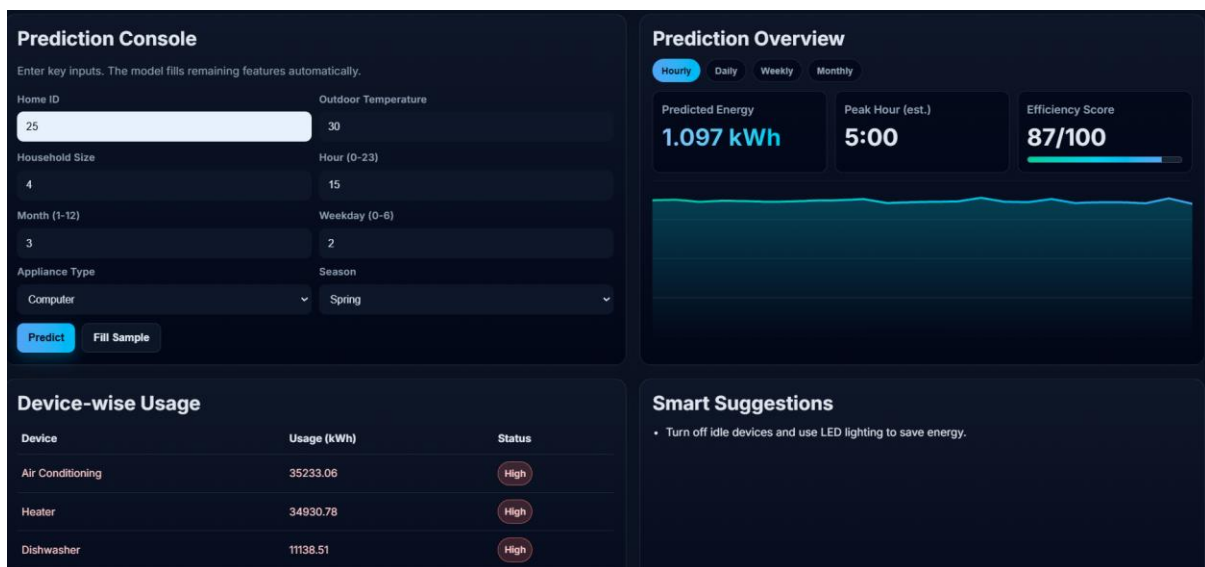
2) Work Completed

Week 7: Backend Integration + Core Features

- Set up Flask app with modular routes for:
 - Home page (/)
 - Prediction API (/api/predict)
 - Usage APIs (/api/usage/hourly, /api/usage/daily, /api/usage/weekly, /api/usage/monthly)
 - Device usage API (/api/usage/devices)
 - Health API (/api/health)
- Loaded trained model from best_energy_model.pkl.
- Implemented feature vector generation from form input:

- Numeric validation and range checks.
- One-hot handling for appliance type and season.
- Added robust model loading with fallback logic (pickle, legacy encoding, joblib).
- Implemented prediction caching (lru_cache) for faster repeated predictions.
- Added logging, request timing, and exception handling.

Week 8: Frontend Redesign + Interactivity



- Redesigned website with modern dark gradient theme and larger typography.
- Increased font sizes across all elements:
- Headings, labels, form fields, table data, KPIs, buttons, suggestions.
- Added rich dashboard interactions:
- Animated counters in hero section.
- Scroll reveal effects.
- Enhanced chart style (gradient line + area fill).
- KPI highlight effects and score progress bar.
- Device status highlighting (High vs Normal).
- Updated UX flow for prediction:
- Loading state (Predicting...)

- Better error display
 - Dynamic KPI refresh after each prediction.
 - Removed non-required content sections as requested:
 - “Why This Project?” section removed.
 - Footer tagline removed.
-

3) Technical Summary

- **Backend:** Python, Flask, NumPy, Pandas
 - **Model:** best_energy_model.pkl (used in live inference via MODEL.predict)
 - **Frontend:** HTML, CSS, JavaScript (vanilla)
 - **Data Handling:** CSV preprocessing and aggregation for analytics APIs
-

4) Testing & Validation

- Verified model file is loaded at startup without runtime failure.
 - Tested form validation for missing/invalid/out-of-range values.
 - Verified API responses for prediction and usage endpoints.
 - Checked dashboard updates for prediction results and suggestions.
 - Confirmed no linter issues in edited frontend files.
-

5) Challenges Faced

- Path/config issues while running app in PowerShell due to incorrect working directory.
 - Python launcher (py) misconfiguration on system.
 - Ensuring UI enhancements did not break existing backend bindings.
 - Aligning feature names with model training schema and fallback mappings.
-

6) Solutions Implemented

- Used explicit absolute path execution for Python where needed.

- Kept model inference flow unchanged while improving frontend only.
 - Added defensive JavaScript checks for optional UI elements.
 - Added robust backend error handling for API and page rendering.
-

7) Outcome

- Delivered a functional and visually improved intelligent energy forecasting web app.
- Prediction pipeline works with uploaded trained model.
- Dashboard is more interactive, readable, and presentation-ready.
- System now better suited for project demo, review, and final submission.